



MoreMarkets Vault

Security Review



MORE

Disclaimer

Security Review

MoreMarkets Vault



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

MoreMarkets Vault

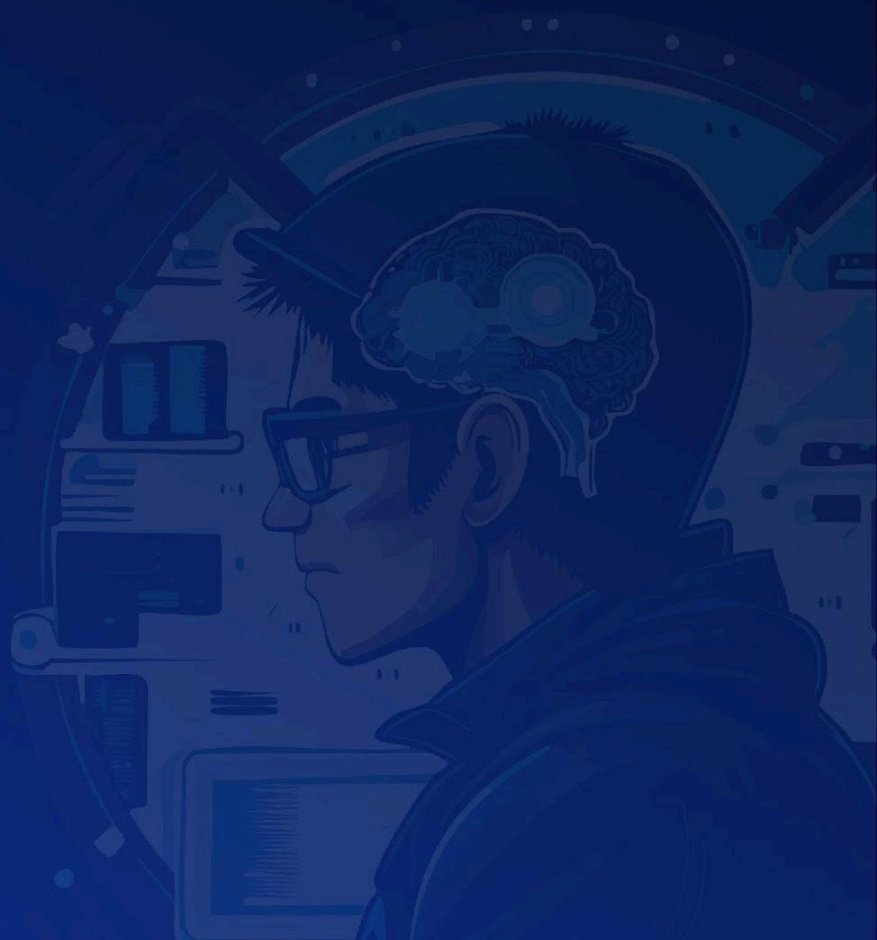


Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Vault-H01	18
3S-Vault-H02	21
3S-Vault-M01	24
3S-Vault-L01	25
3S-Vault-L02	27
3S-Vault-L03	28
3S-Vault-N01	29

Summary Security Review

MoreMarkets Vault



Summary

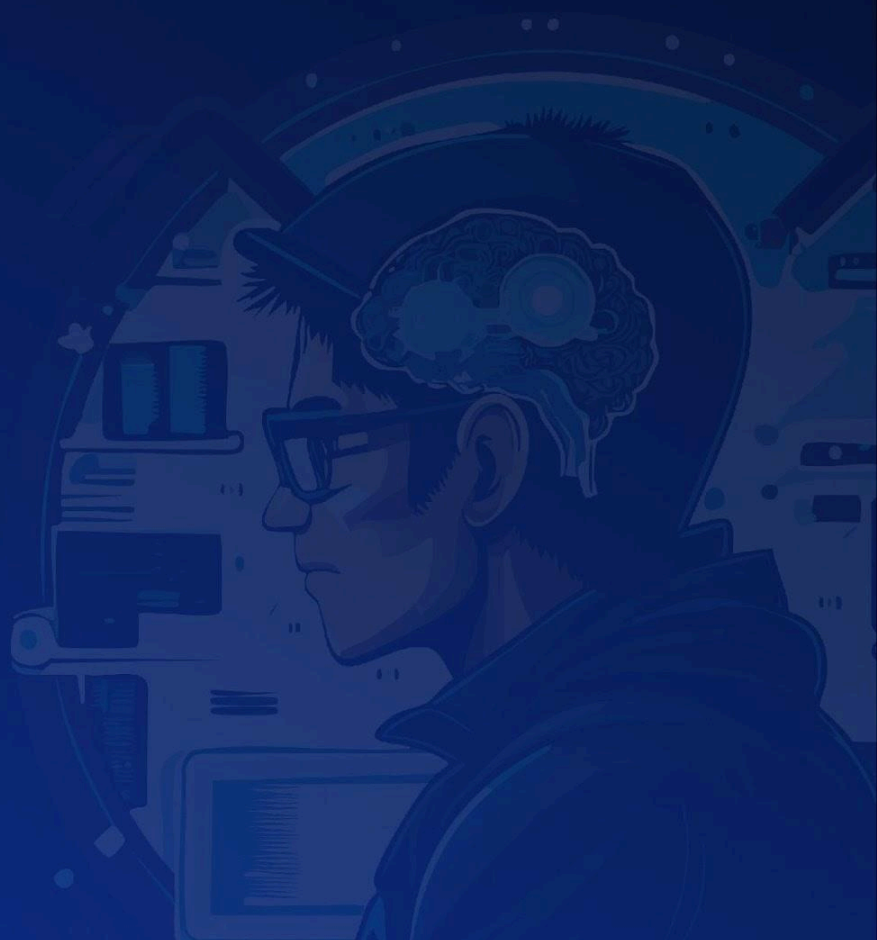
Three Sigma audited MoreMarkets in a 0.8 person week engagement. The audit was conducted from 25/03/2026 to 26/03/2026.

Protocol Description

MORE Vaults is an open-source cross-chain vault standard that combines the composability of the ERC-2535 Diamond Standard with a rigorously audited, invariant and generic core. By separating the asset-critical logic from the ever-evolving strategy layer, MORE drives down implementation and security costs for strategists, integrated protocols, and depositors alike.

Scope Security Review

MoreMarkets Vault

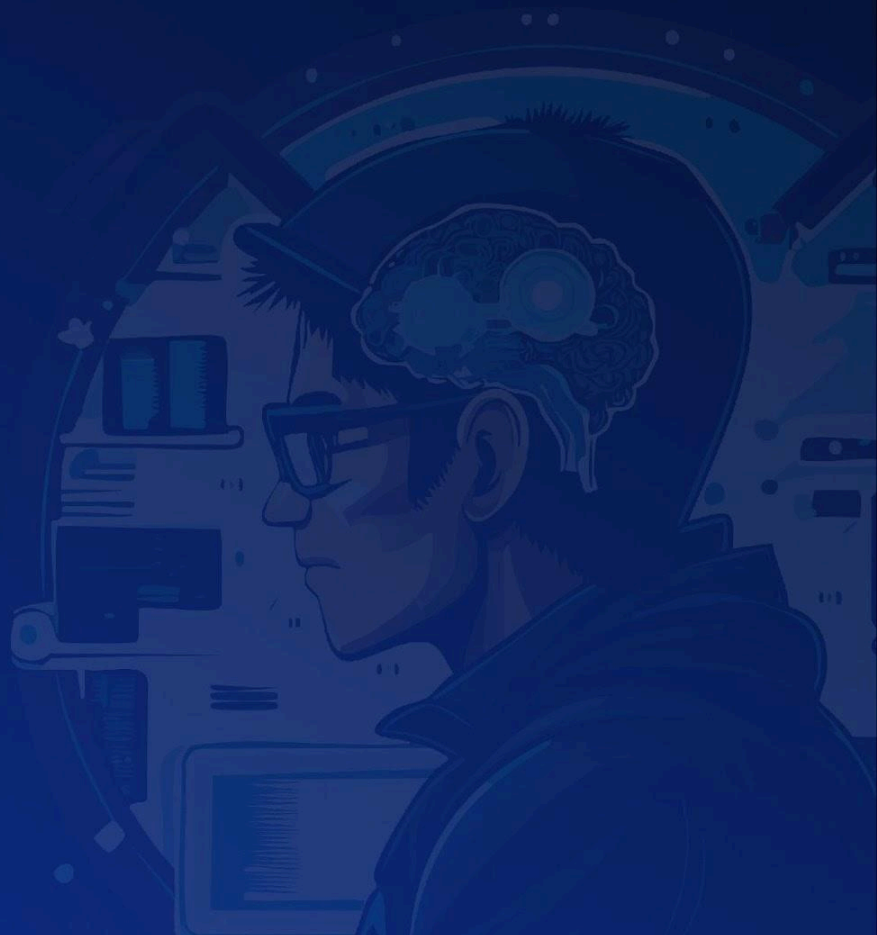


Scope

Filepath	nSL0C
contracts/FlashCollateralSwapReceiver.sol	441
Total	441

Methodology Security Review

MoreMarkets Vault



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

MoreMarkets Vault



Project Dashboard

Application Summary

Name	MoreMarkets
Repository	https://github.com/MOREProtocol/MOREMarket-Allocation-Transfer/
Commit	e4f0d17
Language	Solidity
Platform	Flow

Engagement Summary

Timeline	25/03/2026 to 26/03/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

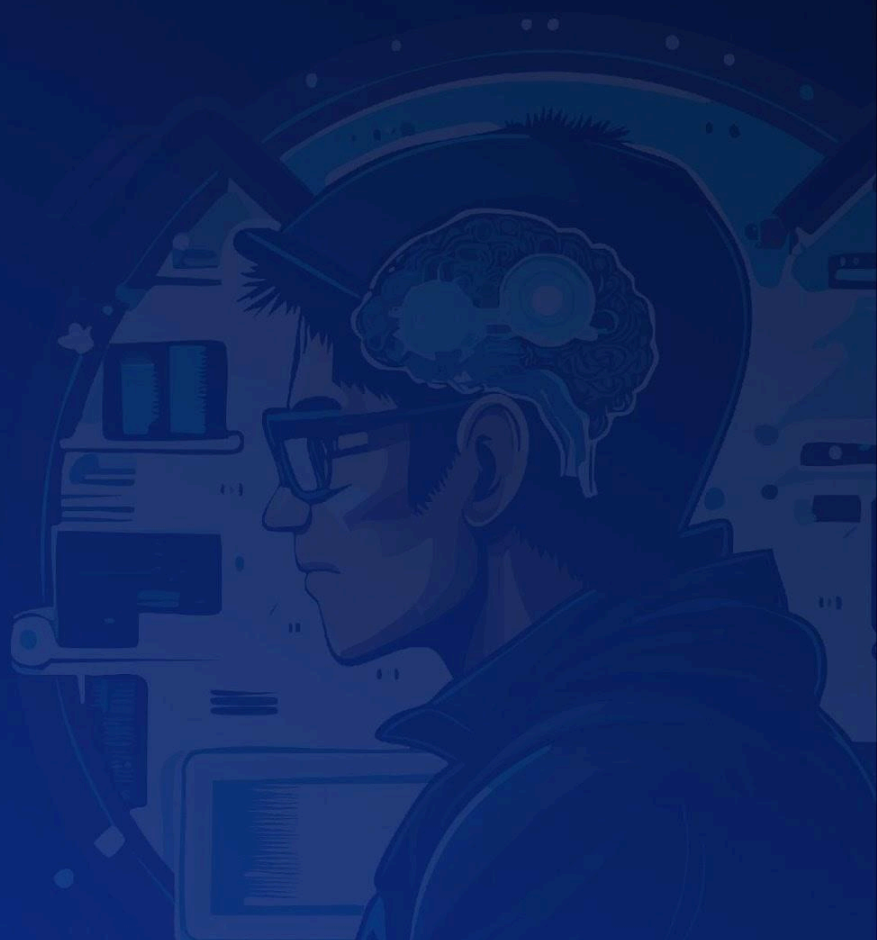
Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	1	1	0
Low	3	3	0
None	1	1	0

Category Breakdown

Suggestion	2
Documentation	0
Bug	4
Optimization	1
Good Code Practices	0

Risk Section Security Review

MoreMarkets Vault



Risk Section

No risks were identified.

Findings

Security Review

MoreMarkets Vault



Findings

3S-Vault-H01

Unvalidated arbitrary calldata in **eisenForwarder** calls allows theft of user-approved aTokens

Id	3S-Vault-H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in e1b6e5d .

Description

FlashCollateralSwapReceiver::executeOperation performs two arbitrary external calls via **_call(cfg.eisenForwarder, ...)** using entirely attacker-controlled targets and calldata. An attacker can exploit these unconstrained calls to steal aTokens from any user who previously approved the receiver contract.

The contract is designed so that legitimate users must approve the receiver to pull their aTokens. Once a user has interacted with the contract or set up an approval for future use, that approval persists. An attacker can abuse the first arbitrary **_call** to execute **transferFrom** on the aToken contract itself, pulling victim tokens directly into the receiver, and then let the contract's own **withdraw(type(uint256).max)** unwrap them into the underlying asset.

The unsafe behavior comes from granting the caller full control over both the external call target (**eisenForwarder**) and the calldata executed by the receiver contract. In practice, this turns the receiver into a generic call proxy that operates with any token approvals previously granted to it by legitimate users.

A realistic attack does not require the attacker to own a meaningful Aave position. A minimal real flash loan is sufficient to enter **executeOperation**, and from there the first arbitrary call can be used to invoke **transferFrom** on a victim-approved aToken contract and pull the victim's aTokens into the receiver. The contract's own **withdraw(type(uint256).max)** logic can then convert those stolen aTokens into the underlying asset, which is later swept to the attacker after flash loan repayment.

The contract-level health factor checks do not mitigate this path because they only evaluate **cfg.user**, which the attacker sets to themselves. As a result, the checks are satisfied as long as the attacker is healthy, while the victim is never validated by this contract. The only remaining bound is Aave's native aToken transfer validation, which merely prevents the victim from being pushed below the protocol's minimum health threshold; it does not prevent theft of any transferable approved balance.

Steps to Reproduce

1. A legitimate user calls **FlashCollateralSwapReceiver::start** to perform a collateral swap. As required by the contract flow, the user has approved the receiver to spend their aWETH. After the operation completes, the approval remains active.
2. The attacker calls **FlashCollateralSwapReceiver::start** with **cfg.user** set to their own address, **cfg.pool** set to the real Aave V3 Pool, **cfg.flashAsset** and **cfg.withdrawAsset** both set to WETH, **cfg.withdrawAToken** set to aWETH, **cfg.eisenForwarder** set to the aWETH contract address, **flashAmount** set to 1, **cfg.depositAsset** set to an unrelated token the receiver does not hold (e.g. DAI), and **cfg.supplyExcessIfFlashIsDeposit** set to false.
3. The attacker encodes **swapFlashToDepositCalldata** as **abi.encodeWithSelector(IERC20.transferFrom.selector, victim, receiverAddress, victimATokenBalance)** and **swapWithdrawToFlashCalldata** as **abi.encodeWithSelector(IERC20.balanceOf.selector, receiverAddress)**.
4. In **executeOperation**, the first **_call** executes **aWETH.transferFrom(victim, receiver, amount)** with **msg.sender = receiver**. Since the victim approved the receiver, the aToken transfer succeeds and the receiver now holds the victim's aWETH.
5. The deposit step is skipped because **depositAsset** balance is zero. The aToken pull from **cfg.user** (attacker) is skipped because the attacker has no aTokens. The **withdraw(type(uint256).max)** burns the receiver's stolen aWETH and sends the equivalent WETH to the receiver.
6. All health factor checks pass because they evaluate the attacker's account (no debt, HF = max). The second **_call** executes a harmless **balanceOf** read. Flash repayment succeeds with 1 wei. **_sweep** sends the victim's underlying WETH to the attacker.

Recommendation

Hardcode the allowed router/forwarder address and remove support for opaque caller-supplied swap calldata. The contract should never forward arbitrary bytes provided by the user while executing in a privileged context with standing token approvals.

Instead, the contract should accept a constrained set of swap parameters such as input token, output token, input amount, minimum output, and any bounded routing metadata, then construct the router calldata on-chain against the known router interface. This ensures

both the call target and the function selector/arguments are under protocol control rather than attacker control.

```
+ address public immutable eisenRouter; // set once in constructor
+
+ constructor(address _eisenRouter) {
+     owner = msg.sender;
+     eisenRouter = _eisenRouter;
+ }
+ // In executeOperation:
- _call(cfg.eisenForwarder, cfg.swapFlashToDepositCalldata);
+ if (cfg.eisenForwarder != eisenRouter) revert InvalidForwarder();
+ bytes memory data = abi.encodeCall(
+     IEisenRouter.swapExactInput,
+     (cfg.flashAsset, cfg.depositAsset, amount, cfg.minDepositOut, cfg.routeData)
+ );
+ _call(eisenRouter, data);
```

Additionally, revoke any temporary token approvals granted to the router at the end of **executeOperation** where feasible, and avoid designs that require users to leave persistent approvals to a highly privileged shared contract.

3S-Vault-H02

Unvalidated pool address allows config injection and theft of all user-approved aTokens

Id	3S-Vault-H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in bb4f96f .

Description

FlashCollateralSwapReceiver::start accepts the pool address as part of the caller-supplied **RebalanceConfig** and passes the entire config through **abi.encode(cfg)** as the **params** argument to **flashLoanSimple**. Because there is no validation that **cfg.pool** points to a legitimate Aave V3 deployment, an attacker can supply a malicious pool contract that replaces the encoded config in the flash loan callback with a forged version where **cfg.user** is set to any victim who has approved the receiver.

The **start** function enforces **msg.sender == cfg.user**, but this only binds the caller to the config they submit. The malicious pool receives the original config in **flashLoanSimple**, discards it, and calls **executeOperation** with entirely different params. Because **executeOperation** decodes its **RebalanceConfig** from the callback **params** rather than from a trusted stored value, the forged config takes effect for the rest of the execution.

Every guard inside **executeOperation** is satisfied by the fake pool. The **NotPool** check compares **msg.sender** against **_activePool**, and since the fake pool is both the stored **_activePool** and the caller, this passes. The **NotInitiator** check is met because the fake pool forwards **address(receiver)** as the initiator argument. The **AssetMismatch** check passes because the fake pool controls both the **asset** parameter and the forged **cfg.flashAsset**.

This makes the callback fully attacker-steerable. In the forged callback, the attacker can substitute a victim as **cfg.user**, point **cfg.pool** to the real Aave Pool for downstream withdrawals, and reuse any standing aToken approvals that legitimate users granted to the receiver as part of normal operation. Once the receiver has pulled the victim's approved aTokens, the rest of the function executes against attacker-chosen state while still satisfying the contract's local guards.

The critical consequence is that the contract mixes trusted execution state from two different sources: the original config used by **start**, and the forged config supplied later by the malicious pool. The fake pool can use the forged config to make the receiver acquire and unwrap victim assets, then return control to **start**, which still uses the original attacker-controlled config for the final sweep. This breaks the intended assumption that the flash loan callback is operating on the same user and pool context that initiated the transaction.

Steps to Reproduce

1. A legitimate user calls **FlashCollateralSwapReceiver::start** to perform a collateral swap and approves the receiver to spend their aWETH as required by the contract flow. The approval remains active after the operation.
2. The attacker deploys a **FakePool** contract that implements **getUserAccountData** (returns **type(uint256).max** as the health factor) and **flashLoanSimple** (calls **receiver.executeOperation** with **amount = 0**, **premium = 0**, **initiator = address(receiver)**, and a forged **RebalanceConfig**). The **FakePool** also includes a **fallback** function that returns successfully for any call, allowing it to serve as a pass-through **eisenForwarder**.
3. The forged config sets **cfg.user** to the victim, **cfg.pool** to the real Aave V3 Pool (so that **withdraw** actually redeems the aTokens), **cfg.eisenForwarder** to the **FakePool** itself (whose fallback absorbs the required **_call** without reverting), **cfg.flashAsset** and **cfg.withdrawAsset** to WETH, and **cfg.withdrawAToken** to aWETH.
4. The attacker calls **FlashCollateralSwapReceiver::start** with **cfg.user** set to their own address and **cfg.pool** set to the **FakePool**. The health factor pre-check calls **FakePool::getUserAccountData** which returns max. The contract sets **_activePool = FakePool** and calls **FakePool::flashLoanSimple**.
5. The **FakePool** calls **receiver.executeOperation** with the forged params. All guards pass: **msg.sender == _activePool** because the fake pool is both, **initiator == address(this)** because the fake pool forwarded the receiver address, and **asset == cfg.flashAsset** because the fake pool controls both values.
6. In **executeOperation**, step 3 pulls all of the victim's aWETH into the receiver via **_safeTransferFrom**. The **withdraw(type(uint256).max)** call goes to the real Aave Pool, burns the receiver's aWETH and sends WETH to the receiver. Health factor checks pass because the victim has no remaining collateral and no debt. The **_call** to **eisenForwarder** hits the **FakePool** fallback which succeeds. Repay passes because **amount + premium = 0**.
7. After **executeOperation** returns, **start** calls **_sweep(WETH, attacker)** using the original config, sending all the victim's underlying WETH to the attacker.

Recommendation

Hardcode the canonical pool address instead of accepting it from user input, and additionally bind the callback to the exact config that initiated the flash loan. Relying only on a config hash is insufficient if the contract still allows callers to choose an arbitrary pool in **start**.

```
+ address public immutable aavePool;
+ bytes32 private _cfgHash;
  constructor(address _aavePool) {
+   owner = msg.sender;
+   aavePool = _aavePool;
+ }
+
  function start(RebalanceConfig calldata cfg, uint256 flashAmount) external {
    // ... existing checks ...
+   if (cfg.pool != aavePool) revert InvalidPool();
+   _cfgHash = keccak256(abi.encode(cfg));
    _activePool = cfg.pool;
    IPool(cfg.pool).flashLoanSimple(...);
+   _cfgHash = bytes32(0);
    _activePool = address(0);
    // ...
  }
  function executeOperation(...) external returns (bool) {
    RebalanceConfig memory cfg = abi.decode(params, (RebalanceConfig));
+   if (keccak256(params) != _cfgHash) revert ConfigMismatch();
    if (msg.sender != _activePool) revert NotPool();
    // ...
  }
```

3S-Vault-M01

Dimensional mismatch in flash amount validation

Id	3S-Vault-M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in e1b6e5d .

Description

The pre-flight check in **start()** compares **flashAmount * ltBps < totalDebtBase * 10_000** to verify sufficient collateral coverage:

```
if (totalDebtBase > 0) {
    uint256 ltBps = _getReserveLT(cfg.depositAsset);
    if (flashAmount * ltBps < totalDebtBase * 10_000)
        revert InsufficientFlashAmountForDebt(
            flashAmount,
            totalDebtBase
        );
}
```

However, **flashAmount** is in token units (decimals vary per asset — 6 for USDC, 8 for WBTC, 18 for WETH) while **totalDebtBase** is in Aave's base currency (USD with 8 decimals). Without oracle price conversion and decimal normalization, the comparison is meaningless. Depending on the token's decimals, the check either always passes (allowing unsafe positions through) or always reverts (blocking valid operations).

Recommendation

Convert **flashAmount** to USD base units using the oracle price and token decimals before comparing, matching what the off-chain script already does. Alternatively, remove the check entirely since the post-operation health factor validation already prevents unsafe positions.

3S-Vault-L01

Absolute health factor floor allows significant collateral loss

Id	3S-Vault-L01
Classification	Low
Impact	Medium
Likelihood	Low
Category	Suggestion
Status	Addressed in bb4f96f .

Description

The contract enforces a fixed minimum health factor of 1.02 (**MIN_HEALTH_FACTOR**). This means a position starting at HF 2.0 could drop to 1.02 and the transaction would still succeed — potentially losing a large portion of the user's collateral value in the process.

```
uint256 internal constant MIN_HEALTH_FACTOR = 1_020_000_000_000_000_000; //
1.02e18
```

In normal operation with correct slippage parameters, such a drastic drop is unlikely. However, if a swap misbehaves due to incorrect slippage settings or a bug in the off-chain script constructing the calldata, the user's position could silently degrade close to liquidation. A fixed floor of 1.02 provides no protection against this scenario — it only prevents outright liquidation.

Recommendation

Add a **maxHfDropBps** field to **RebalanceConfig**, allowing each caller to specify the maximum acceptable HF drop as a percentage of their starting health factor. The check belongs in **executeOperation** alongside the existing HF validation.

```
struct RebalanceConfig {
    // ... existing fields ...
+   uint256 maxHfDropBps;
}
// In executeOperation, after the existing HF check:
+ uint256 minAllowedHf = hfBefore * (10_000 - cfg.maxHfDropBps) / 10_000;
```

```
+ if (hfAfterWithdraw < minAllowedHf) revert  
HealthFactorDropTooLarge(hfAfterWithdraw, minAllowedHf);
```

3S-Vault-L02

No partial rebalance support

Id	3S-Vault-L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Suggestion
Status	Addressed in bb4f96f .

Description

executeOperation always pulls the user's entire aToken balance (line 311), forcing a full collateral swap.

```
uint256 aTokenBal = IERC20(cfg.withdrawAToken).balanceOf(cfg.user);
if (aTokenBal != 0) {
    _safeTransferFrom(cfg.withdrawAToken, cfg.user, address(this), aTokenBal);
}
```

For large positions this can result in significant slippage as sufficient swap liquidity may not be available. Adding a user-controlled amount parameter would allow partial rebalances, enabling users to migrate collateral incrementally with better execution.

Recommendation

Add an **aTokenAmount** field to **RebalanceConfig** and use it instead of the full balance.

3S-Vault-L03

Residual token theft via zero flash loan and fake pool

Id	3S-Vault-L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in 10d588e .

Description

Anyone can call **start** with **flashAmount=0** and a fake pool that simply calls back **executeOperation**. With zero amount and zero premium, all repayment checks pass trivially, and **_sweep** sends the contract's entire residual token balance to the caller. This also creates a race condition with **rescue**- an attacker can front-run the owner's recovery transaction and drain stuck tokens first.

Recommendation

Maintain an owner-managed whitelist of allowed Aave pool addresses.

3S-Vault-N01

Transient storage for flashloan callback state

Id	3S-Vault-N01
Classification	None
Category	Optimization
Status	Addressed in e1b6e5d .

Description

_cfgHash and **_hfBefore** are stored in persistent storage but are only used within a single flashloan callback. Persistent storage costs ~20k gas per first write, while transient storage (EIP-1153) costs ~100 gas per operation and auto-clears at transaction end.

```
bytes32 private _cfgHash;  
uint256 private _hfBefore;
```

Switching both to transient storage saves approximately 40k gas per transaction.

Recommendation

Replace persistent storage with transient storage for **_cfgHash** and **_hfBefore**.